Narrative Quest Generation

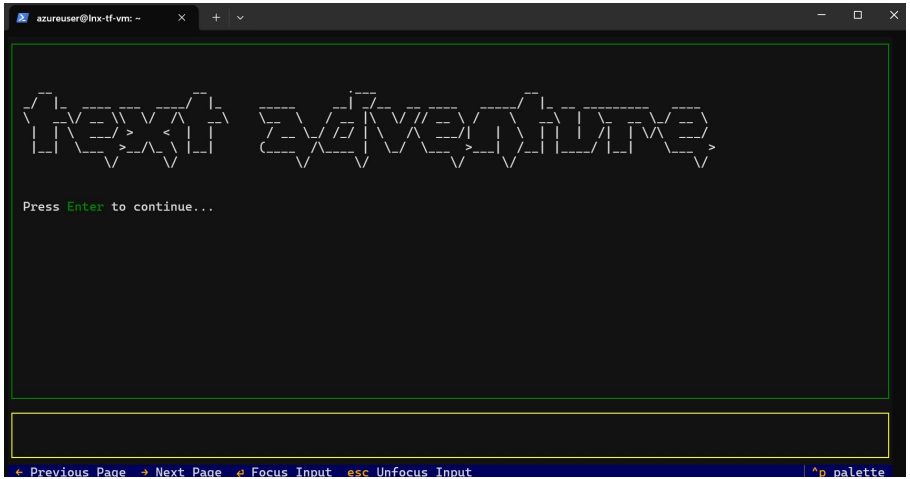
By: Joshua Lester & Caleb Stewart

Project Description

- Text-based adventure assisted by a pretrained Large Language Model (PygmalionAI-12B)
- Think Dungeons and Dragons, but infinite possibilities with the assistance of Deep Learning
- Inspired by games like Zork and AI Dungeon, text-based adventure games where the story evolves dynamically based on the player's actions.
- We're combining language models and vector search to make the game more flexible and immersive.

A Console Application

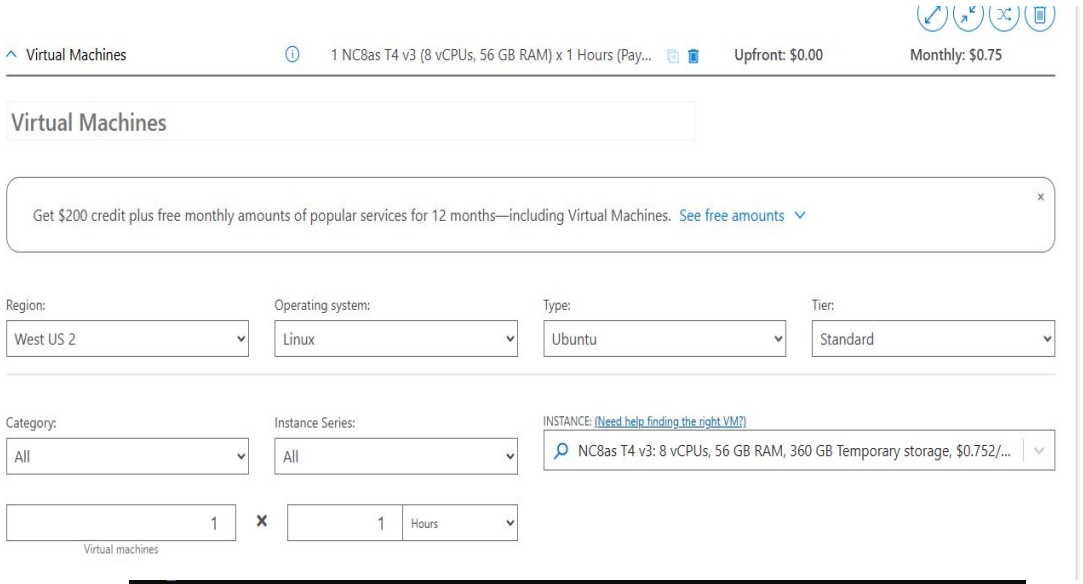
- Built using Python's Textual (a popular TUI framework that allows you to create advanced console applications)
- We chose this option to be able to run the application on a Cloud Virtual Machine (VM) with a powerful GPU to allow for quicker completions
- As shown later, we will SSH into this VM and run the application (a large reason why it's a console app)



```
# Create virtual machine
resource "azurerm_linux_virtual_machine" "lnx-tf-vm" {
  name                  = "lnx-tf-vm"
  location              = azurerm_resource_group.rg.location
  resource_group_name   = azurerm_resource_group.rg.name
  network_interface_ids = [azurerm_network_interface.terraform_nic.id]
  # size                  = "Standard_B2s"
  size                  = "Standard_NC8as_T4_v3"
```

Running in Azure

- We run the application in Azure to allow for GPU acceleration with the LLM
- We used Terraform (infrastructure as code) to automatically allocate and deallocate the VM in Azure quickly
- Also used scripts to allocate the VM, SSH into it, and deallocate it. An additional script is SCPed to make running the application very easy once in the VM




```
dos2unix .\scripts\vm-start.sh

scp -i $key `
    .\scripts\vm-start.sh "azureuser@${ip}:~/vm-start.sh"

ssh -i $key azureuser@$ip

az vm deallocate --name $vm_name --resource-group $rg
```

The Model

- PygmalionAI/Pygmalion-3-12B
  - A model fine tuned for fantasy and roleplaying
- Our chosen Azure VM had 16 GB of GPU RAM
  - The regular model was about 25 GB, so we'd either had to upgrade the VM and pay more, or choose another model
- We discovered model quantization and went with a quantized model instead
  - Model quantization is used to make models smaller and faster, while still maintaining its quality
- We used the GGUF model type to allow for easy access
  - GGUF models are everything you need to run a model stored conveniently in one file (metadata, tensors, architecture info, etc.)
- This model modification was only 12GB, so it could easily be stored in GPU RAM and take advantage of GPU acceleration

How we ran the model

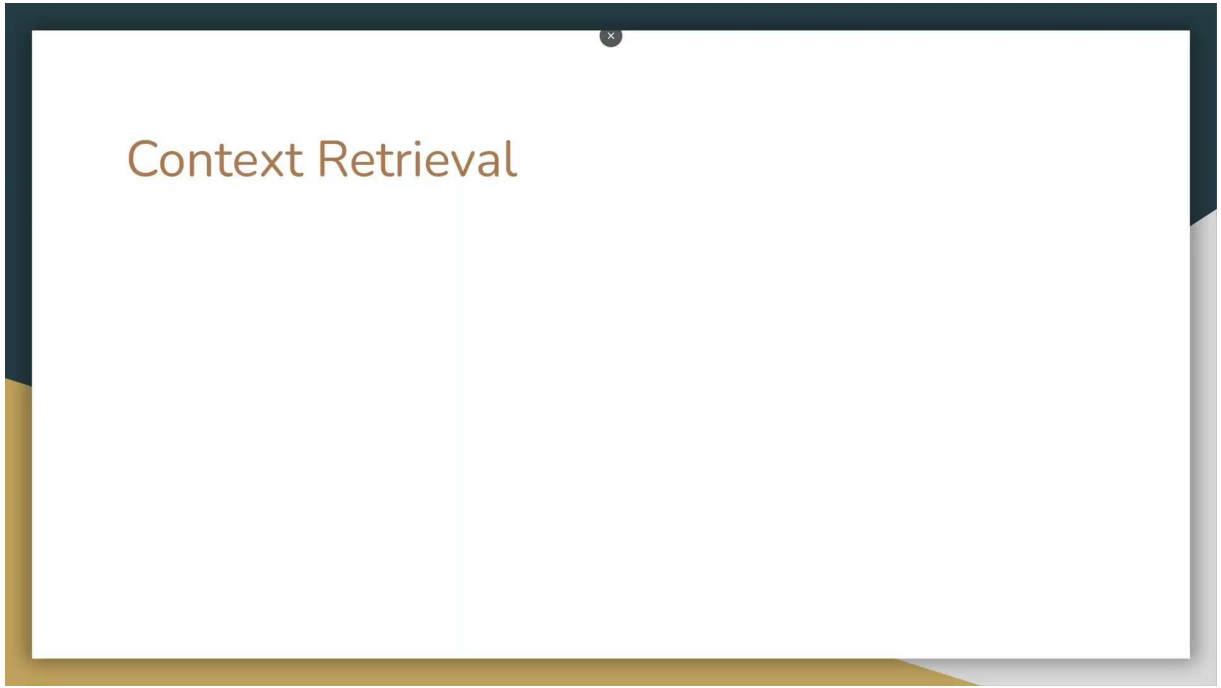
- With a GGUF file, it becomes easy to run the LLM
  - Of course, as long as you have powerful enough hardware and software to support the application
- Ollama fit our needs perfectly
  - Once we ensured the model download through huggingface, we could load the model into an ollama server
  - Next all the app had to do was take user input and generate a response
  - If you pass it in as a parameter, Ollama can take past context and chat much like ChatGPT can do

```
resp = chat(
    model=self.model_alias,
    messages=[*self.chat_history, message],
    options={
        "gpu_layers": 99,  # Use as many layers on GPU as possible
        "temperature": 0.7,
        "top_p": 0.9,
        "num_predict": 2048  # Max tokens to generate
    }
)
```

Prompt Engineering

- While not an impressive form of deep learning utilization, it was our best bet towards directing the LLM where we wanted it to go, without doing expensive fine-tuning
- With context retrieval through vector databases (shown later), we had some bumpers to help the LLM along. We formatted the prompt with relevant info to remind it of past events
- We also maintained an initial prompt to describe the story, and how the LLM should act throughout the story
- Ex: “You are a dungeon master in a fantasy text adventure game. [...] Do not advance time unless the player acts. Never take control of the player’s character. [story…] Begin: ”
- Yet, this approach was still limited. If you got far enough into the story, the LLM would forget important information.

Context Retrieval & FAISS



Intent Extraction

Intent Extraction

Demo

- Switch to terminal to run application